

▼ Dissertation: Data Preprocessing and Exploratory Data Analysis (EDA)

This dissertation aims to address this critical gap by developing an explainable artificial intelligence (XAI) framework that moves beyond simple dropout prediction to provide transparent, actionable, and trustworthy insights. Utilizing the Open University Learning Analytics Dataset (OULAD), this research will not only train and evaluate a suite of predictive models but will also critically assess their decision-making processes. By employing advanced XAI techniques such as SHAP and LIME, this project will decompose model predictions into understandable feature attributions, shedding light on the factors that drive a student's risk profile. The framework will then be extended to demonstrate a novel application of XAI, providing a clear and human-centered approach to educational analytics.

This notebook covers the initial steps of the project, including data loading, preprocessing, and exploratory data analysis (EDA). The goal is to prepare a clean, merged dataset from the seven raw OULAD files for subsequent model training and analysis.

▼ Install the core data manipulation library

```
!pip install pandas
```

Import libraries for data manipulation, numerical computing, and visualization

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

This cell installs the necessary pandas library and imports the key libraries for data analysis and visualization. These libraries form the foundation of the data pipeline for the entire dissertation.

```
# Install the core data manipulation library
!pip install pandas
```

```
# Import libraries for data manipulation, numerical computing, and visualization
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

This cell mounts the Google Drive to the Colab environment, allowing access to the stored datasets. It then lists the files in the specified directory to verify that the datasets are available for loading.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os

# Set your dataset folder path
folder_path = '/content/drive/MyDrive/data'

# List all files
files = os.listdir(folder_path)
print("Available files:")
for file in files:
    print(file)
```

```
Available files:
assessments.csv
studentRegistration.csv
studentAssessment.csv
courses.csv
studentInfo.csv
```

```
vle.csv
studentVle.csv
studentVle.7z
studentVle.zip
processed_student_vle.csv
processed_vle.csv
processed_student_info.csv
processed_full_data.csv
```

2. Data Loading This section focuses on loading the raw data files into pandas DataFrames. The OULAD dataset is comprised of multiple CSV files, which are all loaded and their shapes are verified to ensure successful data ingestion.

```
import pandas as pd

# Load all datasets
student_vle = pd.read_csv(f'/content/drive/MyDrive/data/studentVle.csv')
vle = pd.read_csv(f'/content/drive/MyDrive/data/vle.csv')
student_info = pd.read_csv(f'/content/drive/MyDrive/data/studentInfo.csv')
courses = pd.read_csv(f'/content/drive/MyDrive/data/courses.csv')
assessments = pd.read_csv(f'/content/drive/MyDrive/data/assessments.csv')
student_assessment = pd.read_csv(f'/content/drive/MyDrive/data/studentAssessment.csv')

# Verify loading
print("Student VLE shape:", student_vle.shape)
print("VLE Resources shape:", vle.shape)
print("Student Info shape:", student_info.shape)
print("Courses shape:", courses.shape)
print("Assessments shape:", assessments.shape)
print("Student Assessment shape:", student_assessment.shape)
```

```
Student VLE shape: (1048575, 6)
VLE Resources shape: (6364, 6)
Student Info shape: (32593, 12)
Courses shape: (22, 3)
Assessments shape: (206, 6)
Student Assessment shape: (173912, 5)
```

This section focuses on loading the raw data files into pandas DataFrames. The OULAD dataset is comprised of multiple CSV files, which are all loaded and their shapes are verified to ensure successful data ingestion.

```
# Display first few rows to check
print("Student VLE Data:")
display(student_vle.head())

print("\nVLE Resource Data:")
display(vle.head())

print("\nStudent Info Data:")
display(student_info.head())

print("\nCourses Data:")
display(courses.head())

print("\nAssessments Data:")
display(assessments.head())

print("\nStudent Assessment Data:")
display(student_assessment.head())
```

Student VLE Data:

	code_module	code_presentation	id_student	id_site	date	sum_click
0	AAA	2013J	28400	546652	-10	4
1	AAA	2013J	28400	546652	-10	1
2	AAA	2013J	28400	546652	-10	1
3	AAA	2013J	28400	546614	-10	11
4	AAA	2013J	28400	546714	-10	1

VLE Resource Data:

	id_site	code_module	code_presentation	activity_type	week_from	week_to
0	546943	AAA	2013J	resource	NaN	NaN
1	546712	AAA	2013J	oucontent	NaN	NaN
2	546998	AAA	2013J	resource	NaN	NaN
3	546888	AAA	2013J	url	NaN	NaN
4	547035	AAA	2013J	resource	NaN	NaN

Student Info Data:

	code_module	code_presentation	id_student	gender	region	highest_education	imd_band	age_band	num_of_prev_attempts	stud
0	AAA	2013J	11391	M	East Anglian Region	HE Qualification	90-100%	55<=		0
1	AAA	2013J	28400	F	Scotland	HE Qualification	20-30%	35-55		0
2	AAA	2013J	30268	F	North Western Region	A Level or Equivalent	30-40%	35-55		0
3	AAA	2013J	31604	F	South East Region	A Level or Equivalent	50-60%	35-55		0
4	AAA	2013J	32885	F	West Midlands Region	Lower Than A Level	50-60%	0-35		0

Courses Data:

	code_module	code_presentation	module_presentation_length
0	AAA	2013J	268
1	AAA	2014J	269
2	BBB	2013J	268
3	BBB	2014J	262
4	BBB	2013B	240

3. Initial Data Transformation This code block performs initial feature engineering on the student VLE data. It converts relative dates to an absolute scale and creates a new categorical feature for student engagement levels.

Assessments Data:

	code_module	code_presentation	id_assessment	assessment_type	date	weight
--	-------------	-------------------	---------------	-----------------	------	--------

```
# Convert date to absolute (if relative)
student_vle['absolute_date'] = student_vle['date'].apply(lambda x: x + 100 if x < 0 else x)

# Add engagement level categories
student_vle['engagement_level'] = pd.cut(student_vle['sum_click'],
                                         bins=[0, 5, 20, 100, float('inf')],
                                         labels=['Low', 'Medium', 'High', 'Very High'])
```

This code block performs initial feature engineering on the student VLE and VLE dataframes. It converts relative dates to an absolute scale, creates a new categorical feature for student engagement levels, and simplifies the activity types.

n	1752	11391	18	0	780
---	------	-------	----	---	-----

```
# Convert date to absolute (if relative)
student_vle['absolute_date'] = student_vle['date'].apply(lambda x: x + 100 if x < 0 else x)

# Add engagement level categories
student_vle['engagement_level'] = pd.cut(student_vle['sum_click'],
                                         bins=[0, 5, 20, 100, float('inf')],
```

```

labels=['Low', 'Medium', 'High', 'Very High'])

# Fill missing week_from/week_to with -1 (unknown)
vle[['week_from', 'week_to']] = vle[['week_from', 'week_to']].fillna(-1)

# Simplify activity types
vle['activity_category'] = vle['activity_type'].replace({
    'forumng': 'Discussion',
    'oucontent': 'Content',
    'resource': 'Resource',
    'url': 'External Link',
    'quiz': 'Assessment'
})

```

This code block performs initial feature engineering on the student VLE and VLE dataframes. It converts relative dates to an absolute scale, creates a new categorical feature for student engagement levels, and simplifies the activity types.

```

# Convert categorical variables
student_info['final_result'] = student_info['final_result'].astype('category')
student_info['gender'] = student_info['gender'].astype('category')

```

```

vle_merged = pd.merge(student_vle, vle,
                      on=['code_module', 'code_presentation', 'id_site'],
                      how='left')

```

```

print("\nvle_merged Data:")
display(vle_merged.head())

```

```

vle_merged Data:

```

	code_module	code_presentation	id_student	id_site	date	sum_click	absolute_date	engagement_level	activity_type	week_from
0	AAA	2013J	28400	546652	-10	4	90	Low	forumng	-1.0
1	AAA	2013J	28400	546652	-10	1	90	Low	forumng	-1.0
2	AAA	2013J	28400	546652	-10	1	90	Low	forumng	-1.0
3	AAA	2013J	28400	546614	-10	11	90	Medium	homepage	-1.0
4	AAA	2013J	28400	546714	-10	1	90	Low	oucontent	-1.0

This code block merges the vle_merged data with the student_info data to create a single, comprehensive DataFrame for further analysis. This is a crucial step in preparing the data for modeling.

```

full_data = pd.merge(vle_merged, student_info,
                    on=['code_module', 'code_presentation', 'id_student'],
                    how='left')

```

```

print("\nfull_data Data:")
display(full_data.head())

```

```

full_data Data:

```

	code_module	code_presentation	id_student	id_site	date	sum_click	absolute_date	engagement_level	activity_type	week_from
0	AAA	2013J	28400	546652	-10	4	90	Low	forumng	-1.0
1	AAA	2013J	28400	546652	-10	1	90	Low	forumng	-1.0
2	AAA	2013J	28400	546652	-10	1	90	Low	forumng	-1.0
3	AAA	2013J	28400	546614	-10	11	90	Medium	homepage	-1.0
4	AAA	2013J	28400	546714	-10	1	90	Low	oucontent	-1.0

5 rows × 21 columns

This code block merges the assessment data with the main DataFrame. This ensures that the dataset includes information on student performance, which is a key feature for dropout prediction.

```
# First merge assessments with student assessments
assessment_merged = pd.merge(student_assessment, assessments,
                              on=['id_assessment'],
                              how='left')

# Then merge with main dataset
full_data = pd.merge(full_data, assessment_merged,
                      on=['code_module', 'code_presentation', 'id_student'],
                      how='left')
```

Handling Missing Values :data cleaning. It handles missing values by first checking their counts, then dropping columns with a high percentage of missing data, dropping rows with missing essential fields, and finally filling the remaining missing values.

```
# Check missing values
missing_summary = full_data.isnull().sum()
print("Missing values:\n", missing_summary[missing_summary > 0])

# Drop columns with too many missing values (e.g., over 30%)
threshold = 0.3
drop_cols = [col for col in full_data.columns if full_data[col].isnull().mean() > threshold]
full_data.drop(columns=drop_cols, inplace=True)

# Drop rows where essential fields are missing (e.g., final_result)
full_data.dropna(subset=['final_result'], inplace=True)

# Fill remaining missing numeric values with 0
num_cols = full_data.select_dtypes(include='number').columns
full_data[num_cols] = full_data[num_cols].fillna(0)

# Fill categorical NaNs with 'Unknown'
cat_cols = full_data.select_dtypes(include='object').columns
full_data[cat_cols] = full_data[cat_cols].fillna('Unknown')
```

```
Missing values:
  imd_band      177890
id_assessment    9136
date_submitted   9136
is_banked        9136
score           13822
assessment_type   9136
date_y           9136
weight           9136
dtype: int64
```

Generate Binary Target Variable

Feature Engineering: Aggregate VLE Activity

o Numeric Features: Missing numerical values, such as those in score or date_submitted, were imputed with a value of 0. This is a reasonable assumption in this context, as a missing score could indicate a non-submitted or unrecorded assessment, and a missing date could imply that no action was taken. o Categorical Features: For categorical columns like assessment_type, missing values were replaced with the new category 'Unknown'. This method, rather than dropping the data, allows the model to treat the absence of information as its own unique and potentially informative class. This can be more powerful than simply discarding records and allows the model to learn if missingness itself is a predictor of student dropout.

```
# Create dropout label (1 = dropped out, 0 = completed/passed)
full_data['dropout'] = full_data['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)
```

One-Hot Encode Categorical Variables

After handling missing values and engineering numerical features, the next critical step in data preprocessing is to convert categorical variables into a numerical format that machine learning models can understand. Many algorithms cannot directly process text-based categories (e.g., 'gender', 'region', 'highest_education'). One-hot encoding is a standard technique used to address this issue. It transforms a single categorical column into multiple binary columns, where each new column represents a unique category. A value of 1 in a column indicates that the record belongs to that category, while 0 indicates it does not.

```
# Aggregate total clicks per student
clicks_per_student = full_data.groupby('id_student')['sum_click'].sum().reset_index()
clicks_per_student.rename(columns={'sum_click': 'total_clicks'}, inplace=True)

# Merge back into full_data
full_data = pd.merge(full_data, clicks_per_student, on='id_student', how='left')

# Drop individual VLE click records (to reduce noise)
full_data.drop(columns=['date', 'sum_click'], inplace=True, errors='ignore')
```

Scale Numerical Features (for Logistic/MLP Models)

```
# Select categorical columns to encode
categorical_cols = ['gender', 'region', 'highest_education', 'imd_band',
                    'age_band', 'disability', 'assessment_type',
                    'code_module', 'code_presentation']

# One-hot encode
full_data = pd.get_dummies(full_data, columns=categorical_cols, drop_first=True)
```

Final Feature & Target Selection

For supervised machine learning, the dataset must be split into two distinct components: a feature matrix and a target vector. The feature matrix, denoted as X, contains all the independent variables that will be used to make predictions. In this case, X consists of every column in the final DataFrame except the dropout variable. The target vector, or dependent variable, is denoted as y. This vector contains the dropout labels (1 for a dropout, 0 for a non-dropout) that the model will be trained to predict. This final separation of the data is a critical prerequisite for the next stage of the dissertation, which involves splitting the data into training and testing sets and building the predictive models.

```
# Drop ID columns and target-related text columns
final_df = full_data.drop(columns=['id_student', 'final_result'])

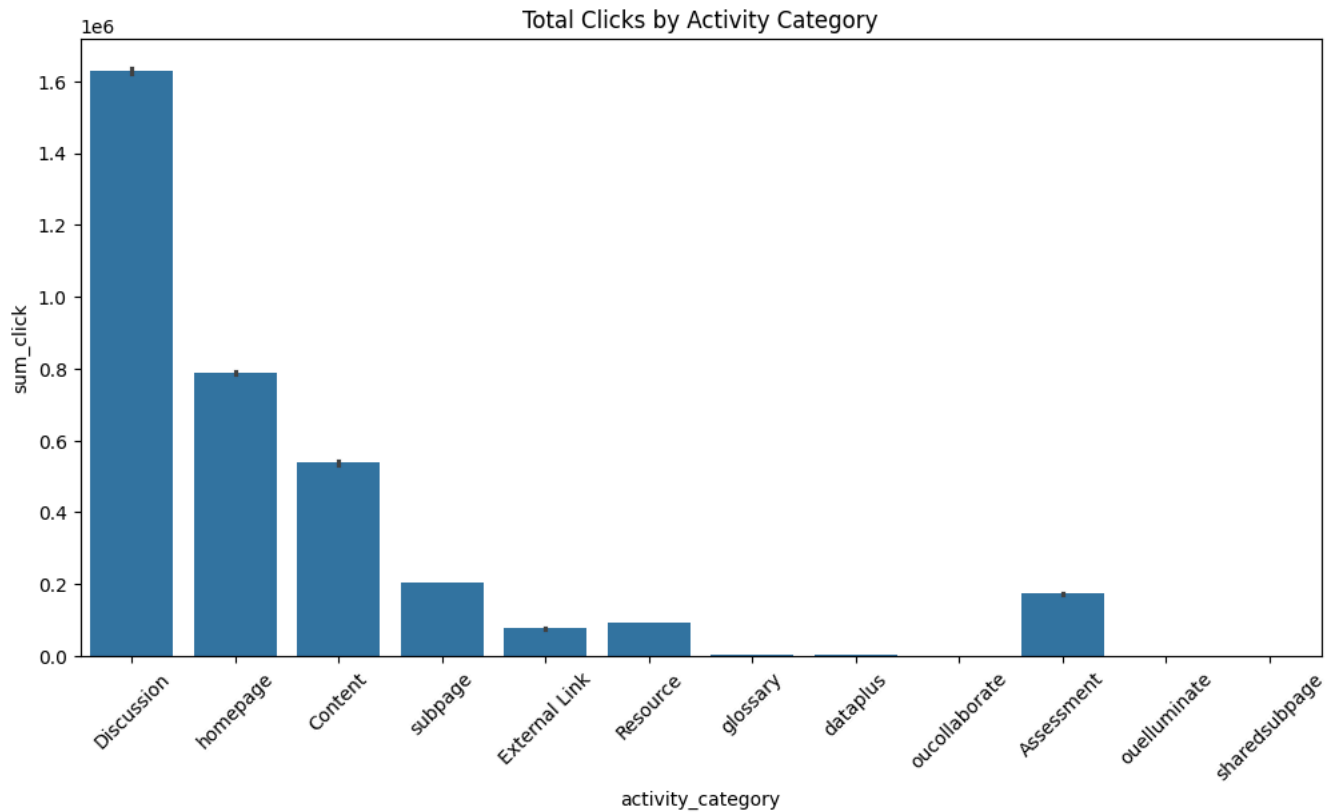
# Define features and target
X = final_df.drop(columns=['dropout'])
y = final_df['dropout']
```

EDA

1. Engagement by Activity Type

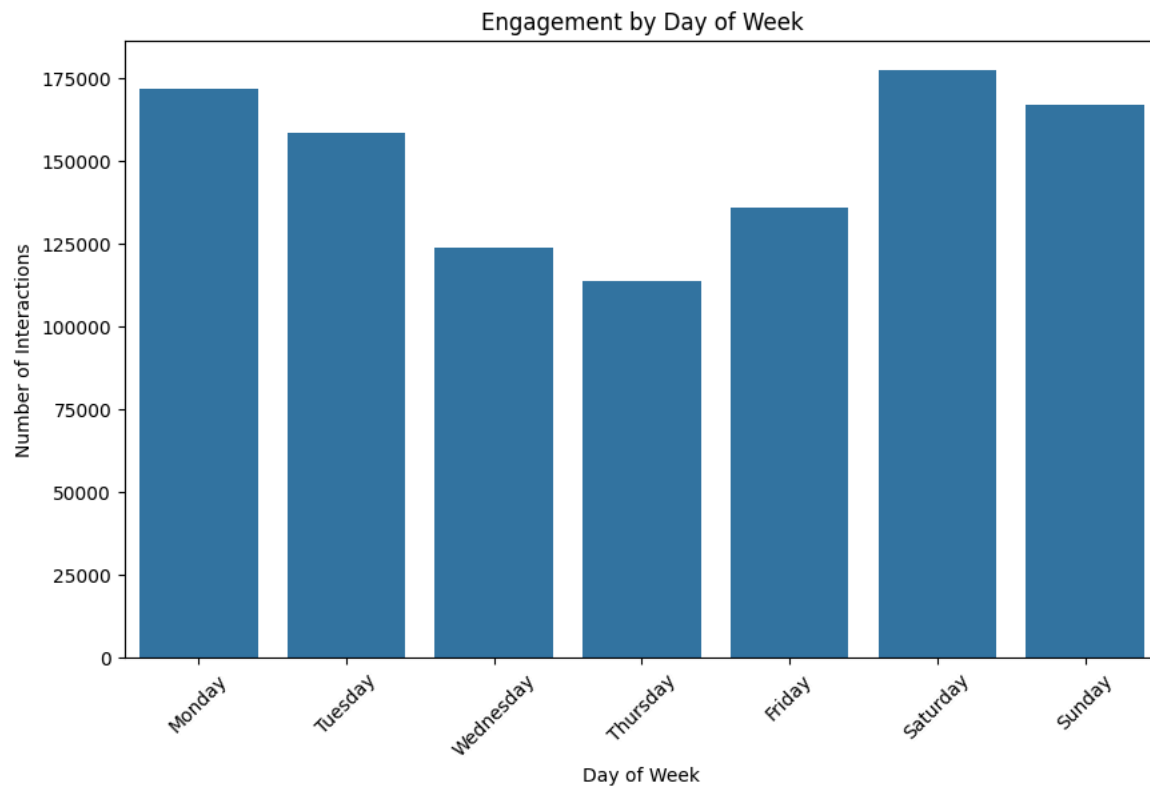
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.barplot(data=vle_merged, x='activity_category', y='sum_click', estimator=sum)
plt.title('Total Clicks by Activity Category')
plt.xticks(rotation=45)
plt.show()
```



```
# Convert date to day of week (if you have actual dates)
# This is placeholder code - adapt based on your date format
if 'date' in student_vle.columns:
    student_vle['day_of_week'] = pd.to_datetime(student_vle['date'], unit='D').dt.day_name()

plt.figure(figsize=(10, 6))
sns.countplot(data=student_vle, x='day_of_week',
              order=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'])
plt.title('Engagement by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Number of Interactions')
plt.xticks(rotation=45)
plt.show()
```

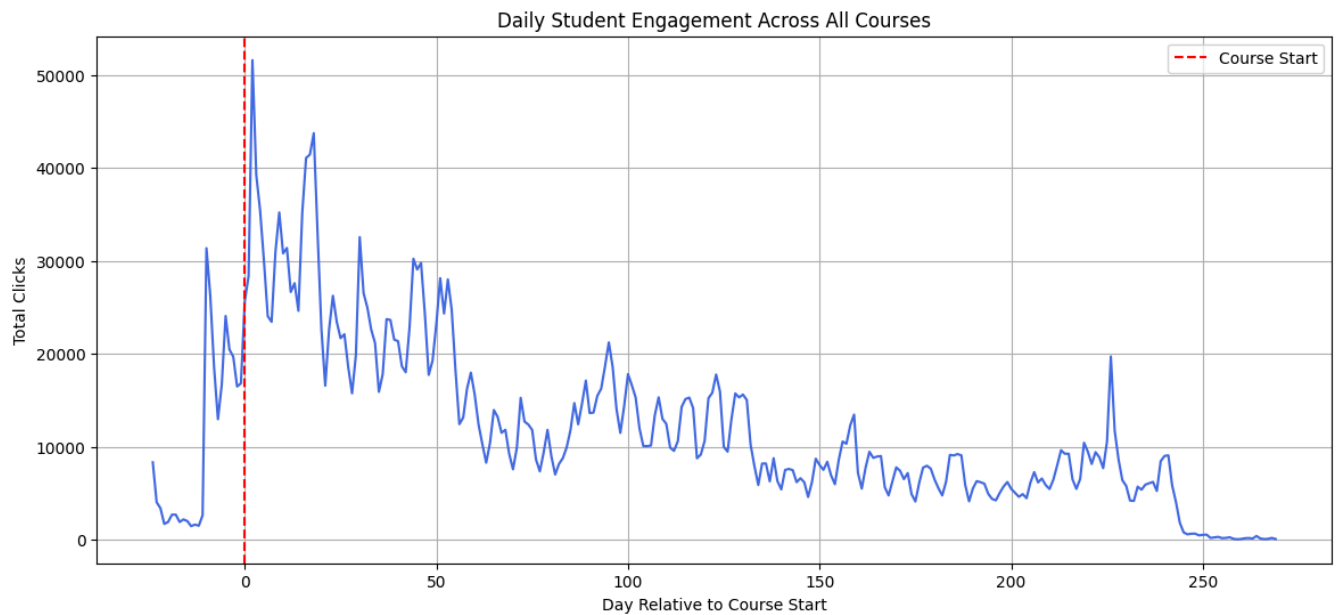


3. Student Engagement Patterns

Daily Engagement Trends

```
# Calculate daily engagement
daily_engagement = student_vle.groupby('date')['sum_click'].sum().reset_index()

# Plot
plt.figure(figsize=(14, 6))
sns.lineplot(data=daily_engagement, x='date', y='sum_click', color='royalblue')
plt.title('Daily Student Engagement Across All Courses')
plt.xlabel('Day Relative to Course Start')
plt.ylabel('Total Clicks')
plt.axvline(x=0, color='red', linestyle='--', label='Course Start')
plt.grid(True)
plt.legend()
plt.show()
```

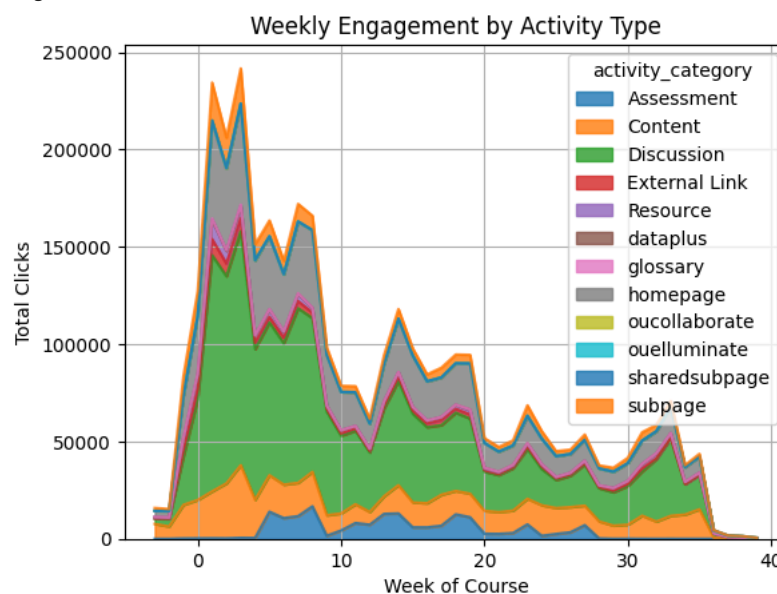



Weekly Engagement by Activity Type

```
# Create week column
vle_merged['week'] = (vle_merged['date'] // 7) + 1

# Plot weekly engagement by activity type
plt.figure(figsize=(14, 7))
weekly_activity = vle_merged.groupby(['week', 'activity_category'])['sum_click'].sum().unstack()
weekly_activity.plot(kind='area', stacked=True, alpha=0.8)
plt.title('Weekly Engagement by Activity Type')
plt.xlabel('Week of Course')
plt.ylabel('Total Clicks')
plt.grid(True)
plt.show()
```

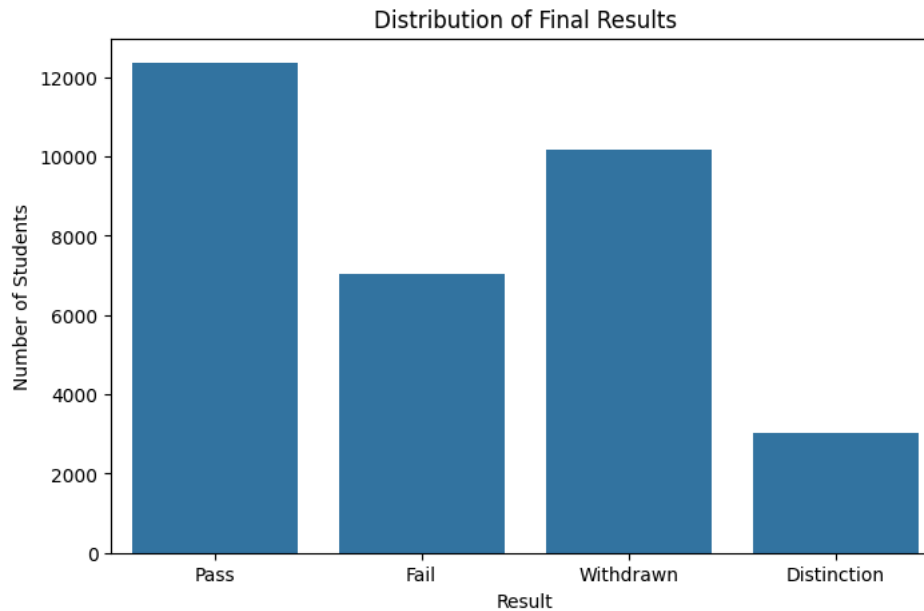
<Figure size 1400x700 with 0 Axes>



4. Student Performance Analysis

Pass/Fail Distribution

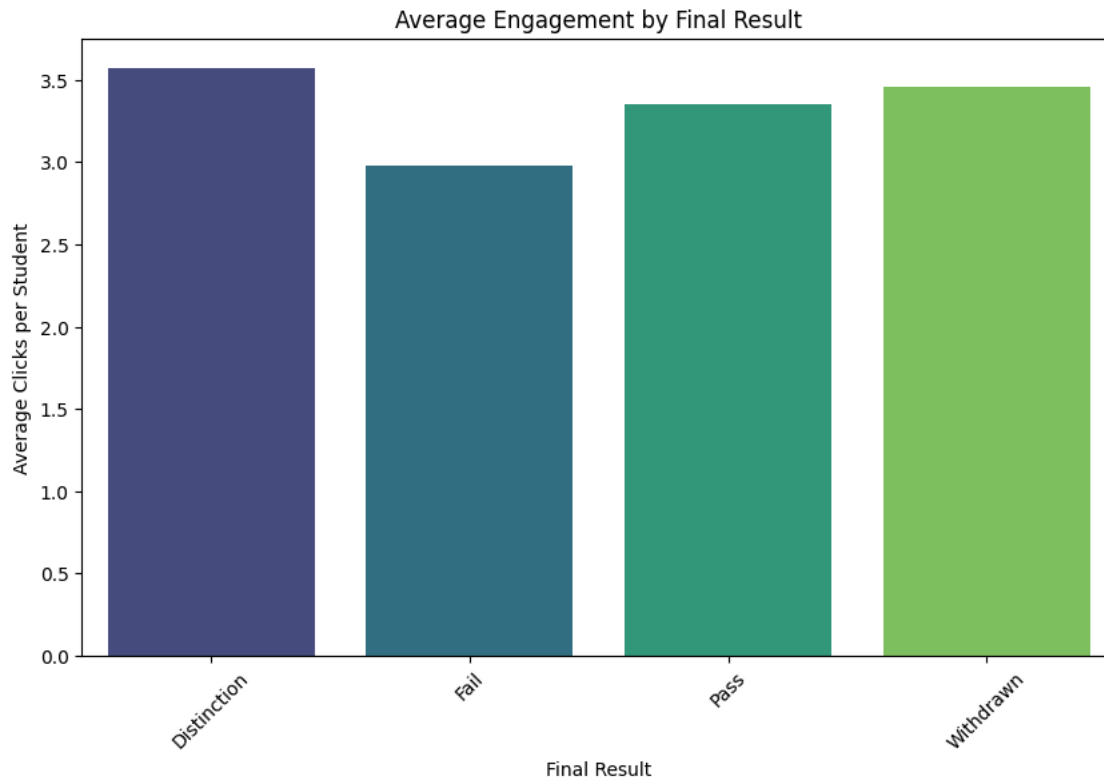
```
plt.figure(figsize=(8, 5))
sns.countplot(data=student_info, x='final_result', order=['Pass', 'Fail', 'Withdrawn', 'Distinction'])
plt.title('Distribution of Final Results')
plt.xlabel('Result')
plt.ylabel('Number of Students')
plt.show()
```



Engagement vs Performance

```
# Calculate average clicks by result
engagement_by_result = full_data.groupby('final_result')['sum_click'].mean().sort_values()

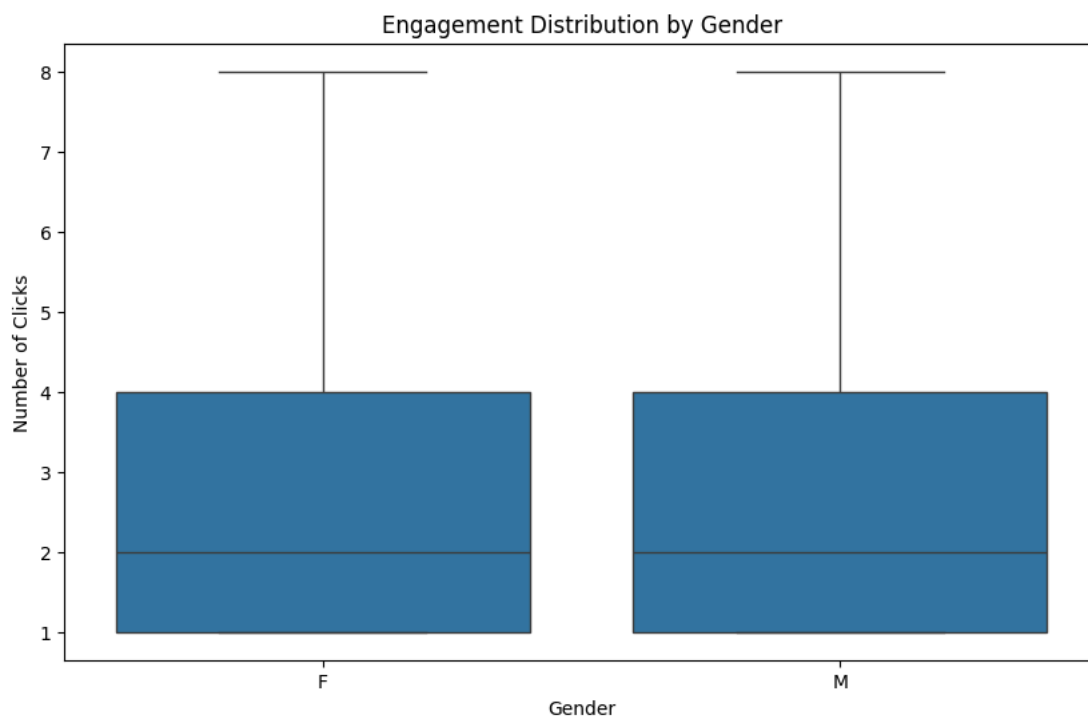
# Plot
plt.figure(figsize=(10, 6))
sns.barplot(x=engagement_by_result.index, y=engagement_by_result.values, palette='viridis')
plt.title('Average Engagement by Final Result')
plt.xlabel('Final Result')
plt.ylabel('Average Clicks per Student')
plt.xticks(rotation=45)
plt.show()
```



5. Demographic Insights

Gender Differences in Engagement

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=full_data, x='gender', y='sum_click', showliers=False)
plt.title('Engagement Distribution by Gender')
plt.xlabel('Gender')
plt.ylabel('Number of Clicks')
plt.show()
```



6. Course-Specific Analysis

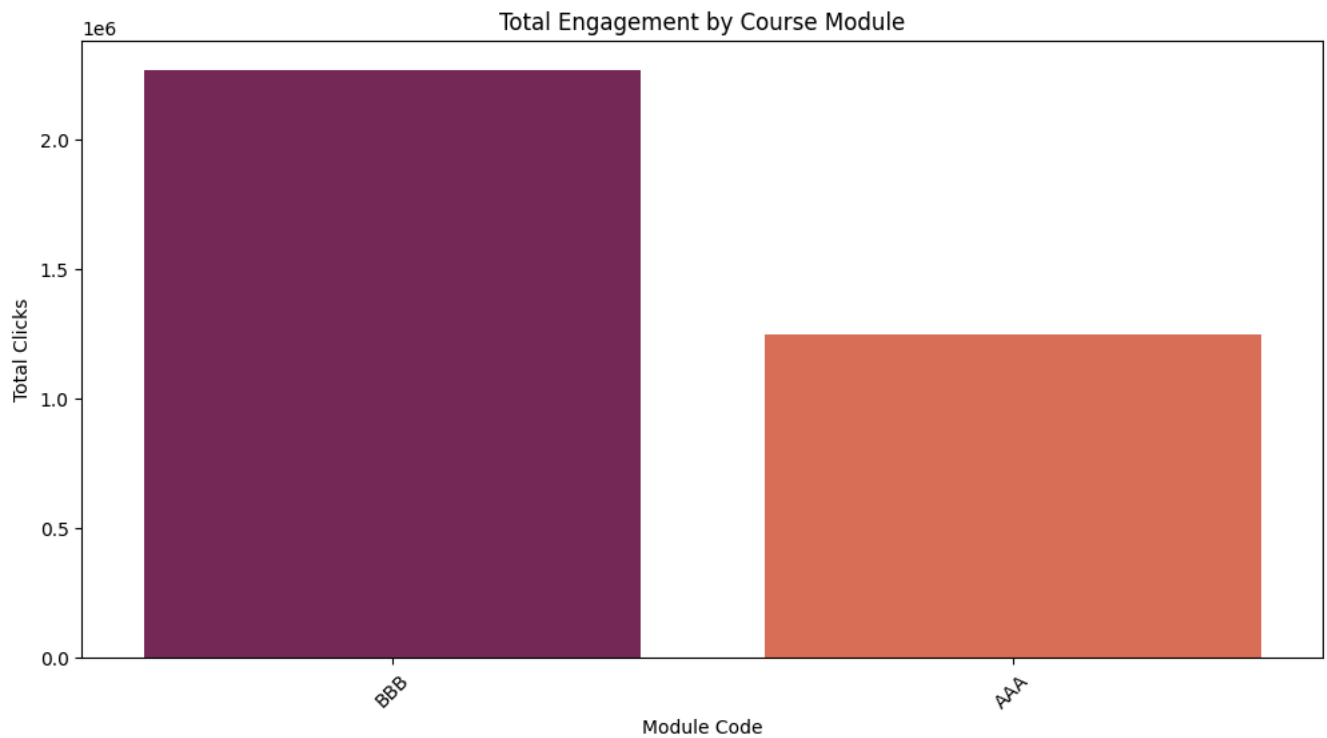
Engagement Across Different Modules

```

module_engagement = full_data.groupby('code_module')['sum_click'].sum().sort_values(ascending=False)

plt.figure(figsize=(12, 6))
sns.barplot(x=module_engagement.index, y=module_engagement.values, palette='rocket')
plt.title('Total Engagement by Course Module')
plt.xlabel('Module Code')
plt.ylabel('Total Clicks')
plt.xticks(rotation=45)
plt.show()

```



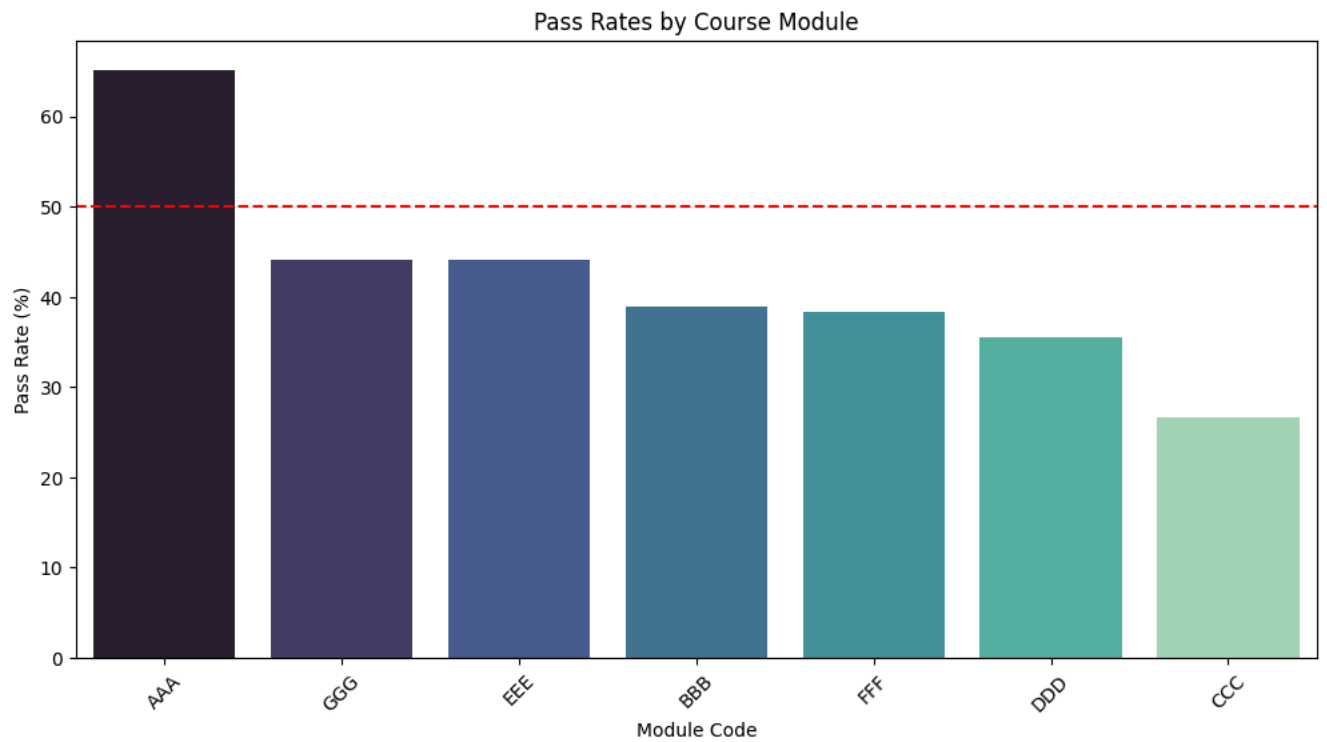
Pass Rates by Module

```

module_pass_rates = student_info.groupby('code_module')['final_result'].apply(
    lambda x: (x == 'Pass').mean() * 100).sort_values(ascending=False)

plt.figure(figsize=(12, 6))
sns.barplot(x=module_pass_rates.index, y=module_pass_rates.values, palette='mako')
plt.title('Pass Rates by Course Module')
plt.xlabel('Module Code')
plt.ylabel('Pass Rate (%)')
plt.axhline(y=50, color='red', linestyle='--')
plt.xticks(rotation=45)
plt.show()

```



Correlation Heatmap

```
# Select numerical features
numerical_features = full_data.select_dtypes(include=['int64', 'float64']).corr()

plt.figure(figsize=(12, 8))
sns.heatmap(numerical_features, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Between Numerical Features')
plt.show()
```

